



# Concurrency control generation for dynamic threads using discrete-event systems



Anthony Auer<sup>a</sup>, Juergen Dingel<sup>a,\*</sup>, Karen Rudie<sup>b</sup>

<sup>a</sup> School of Computing, Queen's University, Kingston, Ontario, K7L 3N6, Canada

<sup>b</sup> Department of Electrical and Computer Engineering, Queen's University, Kingston, Ontario, K7L 3N6, Canada

## ARTICLE INFO

### Article history:

Received 26 April 2012

Received in revised form 18 December 2012

Accepted 25 January 2013

Available online 8 February 2013

### Keywords:

Programming techniques  
Concurrent programming  
Software engineering  
Design tools and techniques  
Petri nets  
Discrete-event systems  
Concurrency control  
Autogeneration  
Limited lookahead

## ABSTRACT

The application of Discrete-Event Systems (DES) theory to the problem of guaranteeably enforcing concurrency constraints in multithreaded applications has been studied under certain assumptions, namely, the assumption of a static pool of pre-existing instantiated threads, whose creation and termination are not modeled. This work presents an extension of this case to handle dynamically instantiated and terminated threads using a Petri net formalism and an online limited-lookahead state-space search technique.

© 2013 Elsevier B.V. All rights reserved.

## 1. Introduction

Concurrent or *multithreaded* software design and development is an ever-increasing priority for study and practice. Aside from hardware and performance considerations, concurrent software design can be indicated when the problem itself is naturally expressible as multiple independent processes, or when the individual subproblems are actually solved in an intrinsically remote manner, possibly on different hardware altogether. Despite its benefits, execution remains very difficult for most practitioners; aggregate thread behavior is considerably more complex than that of a single thread.

The work of Dragert et al. [1] has demonstrated the use of discrete-event systems (DES) to model a static set of concurrently executing threads in a high-level programming language, automatically generating source code which imposes control according to globally-specified constraints. Such control (called *supervisory control* in the DES context) has several guaranteeable characteristics, most notably deadlock freedom and *maximal permissiveness* (allowing the largest subset of possible behavior under the constraints). The net effect is to free the developer from having to manually devise correct concurrency control code, increasing productivity and software quality.

A central assumption of Dragert et al. is that thread lifetime is static—their creation and termination are not modeled—an assumption that presents a barrier to real-world application. This work sets out to accomplish two goals: first, to accommodate the modeling of dynamic threads in a way in which their creation and termination is part of the action of

\* Corresponding author.

E-mail addresses: [auer.anthony@gmail.com](mailto:auer.anthony@gmail.com) (A. Auer), [dingel@cs.queensu.ca](mailto:dingel@cs.queensu.ca) (J. Dingel), [karen.rudie@queensu.ca](mailto:karen.rudie@queensu.ca) (K. Rudie).

the system under control<sup>1</sup>; second, to maintain the tractability of the control decision in the face of a large state space which cannot be computed in its entirety offline.

The first goal is accomplished through the use of Petri nets, a family of automata which inherently reflects concurrent state transitioning. Petri nets have been used to model resource flow in complex manufacturing systems for decades [2], and the literature abounds with discourse on their use in a variety of applications. Attention is restricted here to a subclass of Petri nets which reflect the structure of a concurrent software application. Petri nets are used elsewhere for concurrency control applications, most notably the work of Wang et al. [3]; the principal difference between their work and ours is their focus on analysis of the static Petri net structure rather than on online computation of Petri net transitions.

The second goal is approached by adopting an online search strategy (one which is computed during, rather than prior to, software execution) coupled with a limited-lookahead search [4]. Limited lookahead refers to an incomplete search of future execution sequences, relying on a policy to heuristically fill in the information gap due to those areas left unsearched.

As an extension to the earlier static case, the dynamic approach represents an increase in generality over the regular language capability inherent in the finite state-based approach of Dragert et al. In particular, non-context-free languages can be specified by Petri nets [5]. With a further extension of Petri nets (called *extended Petri nets*), expression of arbitrary constraints is achieved, but at a cost: the limited lookahead technique guarantees an upper bound on computation time, but the control decision computation must be performed online (during execution), meaning that it must be very quick to avoid performance compromises. Moreover, because the state-space exploration is not complete, there is the possibility of precision loss. This loss is more fully treated in Section 3.4.

Control theory has recently been attracting growing attention from researchers interested in software development. The idea is to use control theory to control the execution of software and leverage the existing body of work (such as theoretical results and analysis and synthesis approaches and tools) to control complexity and increase software reliability and development productivity [6]. For instance, recent proposals advocate the use of control theory for different kinds of systems including adaptive systems [7], and mission-critical systems [8,9]. This paper extends our previous work on using discrete-event systems theory (or, discrete-event control theory) for the implementation of concurrent systems in [1] and is thus similar in spirit to the work of the Gadara project [10,11].

The structure of the paper is as follows: Relevant concepts from discrete-event systems theory are reviewed in Section 2, while Section 3 describes how Petri nets are used to model concurrent code. Sections 4–6 present the heart of our approach by discussing, respectively, the supervisor lookahead algorithm, the online supervisor synthesis, and the supervisor's runtime behavior. Implementation and initial validation are described in Section 7, followed by a detailed discussion of related work in Section 8. Section 9 concludes and sketches some future work.

## 2. Discrete-event systems

Discrete-event systems theory models a system under control as a set of states traversed via sequences of (possibly nondeterministic) occurrences of instantaneous events. Though there exist many variations and embellishments, the original Ramadge–Wonham discrete-event system framework is generally represented by *finite state automata* or FSAs [12] with additional features to represent concepts of control. More details on the automata-theoretic concepts that we use can be found in [12]. The salient features are a *state-space*  $Q$ , an *alphabet*  $\Sigma$  of symbols, a *transition function*  $\delta : Q \times \Sigma \rightarrow Q$  which takes a state-symbol tuple and produces another state and a recursive transition function  $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$  defined such that

$$\begin{aligned}\hat{\delta}(q, \epsilon) &= q \\ \hat{\delta}(q, s\alpha) &= \delta(\hat{\delta}(q, s), \alpha)\end{aligned}$$

where  $\Sigma^*$  is the set of *strings* of elements of  $\Sigma$ ,  $\epsilon$  the “empty” string,  $\alpha \in \Sigma$ ,  $s \in \Sigma^*$  and  $q \in Q$ . The alphabet of labels is viewed as a collection of possible instantaneous events (called the *event set*) whose occurrence can effect state transitions (as per the transition function  $\delta$ ). A sequence of zero or more such state transitions is known as a *trace*; the (unique) sequence of zero transitions is known as the *empty trace*. In addition, the property of *controllability* partitions the alphabet into a cover of two disjoint subsets: *controllable* events  $\Sigma_c \subseteq \Sigma$  and *uncontrollable* events  $\Sigma_u = \Sigma \setminus \Sigma_c$ . We imagine that, there is a process whose behavior is unsatisfactory. Such a process is called the *plant* (a term borrowed from classical control theory). The plant is modeled as an automaton generating event sequences. The plant (typically denoted  $G$ ) may be modeled as a monolithic automaton, or as a set of automata, the synthesis of whose actions comprises the complete system. The plant generates a language  $\mathcal{L}(G)$ ; a subset of the states of  $G$  can be *marked* (in the usual sense of accepting states described in [12]), informing the *marked sublanguage*  $\mathcal{L}_m(G) \subseteq \mathcal{L}(G)$ .

Discrete-event systems make extensive use of operations definable on regular languages and their analogues on finite state automata [13], such as *parallel composition* (traversal event-by-event in lock-step for shared events and interleaved for unshared) and *shuffle* (all possible asynchronous interleavings of event sequences).

<sup>1</sup> We consider a system  $M$  to support *dynamic concurrency* or have *dynamic threads* if it allows thread creation and destruction at runtime and the number of threads in  $M$  can grow and shrink during the execution of  $M$  in a way that need not and sometimes even cannot be known prior to runtime.

The object of control is to impose a set of desired constraints upon the behavior of the plant, which is accomplished by partitioning  $\mathcal{L}(G)$  by *legality*. Legality is generally expressed using one or more user-created *specifications*, denoted by  $S_i$ ; where the specification is a regular language, the constraints can be represented as a collection of finite automata. The overall *legal language*  $E$  is the language resulting from the intersection of all the specifications  $S_i$ .

**Definition 2.1.** Let  $Q$  and  $\Sigma$  be, respectively, the state-space and the alphabet of a plant  $G$ . An event  $\alpha \in \Sigma$  is *legal* at a state  $q \in Q$  if each specification  $S_i$  allows the event to occur—i.e., it is enabled—from  $S_i$ 's current state.

An extension of the notion of legality from events to traces follows:

**Definition 2.2.** The empty trace is legal. If  $q_0$  is the initial state of an automaton  $M$ , a trace of length  $n > 0$  is legal in  $M$  if its prefix  $s$  of length  $n - 1$  is legal and its suffix of length 1 (whose equivalence to an event we take as defined) is legal with respect to the state  $\hat{\delta}(q_0, s)$ .

If we define  $E$  as the language of all legal traces in  $G$ , the foregoing is expressible as:

$$\epsilon \in E$$

$$s\alpha \in E \Leftrightarrow s \in E \wedge \alpha \in E/s.$$

The automaton sought out is a *supervisor*  $S$  which, in intersection with the plant, yields an automaton  $S/G$  (called the *closed-loop system*) the generated *closed-loop language*  $\mathcal{L}(S/G)$  of which has the properties that all its sequences satisfy the specification  $E$  and is closed with respect to  $\Sigma_u$  (that is, any extension via uncontrollable events must also be in  $\mathcal{L}(S/G)$  - it cannot restrict the occurrence of any uncontrollable subsequences generable by  $G$ ):

$$\mathcal{L}(S/G) \subseteq E \cap \mathcal{L}(G) \quad \text{and} \quad s \in \mathcal{L}(S/G) \wedge \sigma \in \Sigma_u \Rightarrow s\sigma \in \mathcal{L}(S/G).$$

Moreover,  $\mathcal{L}(S/G)$  must be *nonblocking*: any event sequence generable by the system must be a *prefix*<sup>2</sup> of a sequence marked by it (i.e.,  $\mathcal{L}(S/G) \subseteq \overline{\mathcal{L}_m(S/G)}$ ).

Ramadge and Wonham [14] show that such subsets of  $E \cap \mathcal{L}(G)$  with the above properties form an upper semilattice partially ordered by set inclusion; thus there is a largest such subset, called the *supremal controllable sublanguage* of  $E$  with respect to  $G$ , and denoted  $\text{supC}(E)$  (or  $\text{supcon}$  in the parlance of prevalent Discrete-Event Systems software such as TCT [15] or IDES [16]). Moreover, they showed that such a language (and a supervisor  $S$  required to enforce it) is algorithmically derivable in  $O(|E|^2|G|^2)$  time [17] (where the cardinality notation  $|G|$  denotes the number of states of automaton  $G$  and  $|E|$  denotes the number of states in a minimal-state DFA that recognizes  $E$ ).<sup>3</sup> This result not only satisfies the specification, but is also both nonblocking by construction and maximally permissive by definition of supremum.

Dragert et al. [1] provide a method for using specifications expressed directly as automata to automatically generate control code in a given high-level language (the *target language*). To do so, a transformation of the target code to an automaton must first be carried out; this is achieved through the use of instrumentation of relevant events—those related to the truth of the specification. The automaton transformations described above (parallel composition,  $\text{supcon}$ ) are then used to construct the appropriate finite state automaton which acts as a supervisor; this automaton can then be used to generate a state machine and control code in the target language.

Apart from eliminating the need to manually design concurrency control, the principal advantage of the approach of Dragert et al. is the mathematical guarantee of deadlock freedom it provides due to the use of  $\text{supcon}$  to find the supremal controllable sublanguage; in addition, by the definition of the supremal controllable sublanguage, it guarantees maximal permissiveness. The deadlock-freedom guarantee offers an advantage over earlier works, in which the latter had to be checked using model verification techniques [18].

By way of motivation for the stated problem we introduce a simple concurrency problem derivative of the so-called *Readers/Writers problem* [19]. The problem involves a set of autonomous reader and writer processes which access a single shared resource, the integrity of which must be preserved. Skeletons of such processes are depicted by the Java code in Fig. 1.

The Readers/Writers problem demonstrates an *asymmetrical mutual exclusion (mutex)* constraint: while many readers may read simultaneously, the shared resource may only be written by one writer at a time. The critical sections in the example code are comprised of the **read()** and **write()** functions of the reader and writer, respectively. The goal of supervisory control in this example is to prevent any **write()** that is simultaneous with other **read()** and **write()** calls, while allowing simultaneous **read()** calls.

The FSA representations of Readers and Writers are depicted in Fig. 2. As mentioned, the method of Dragert et al. assumes a pre-set, static number of Readers and Writers, the synthesis of which by parallel composition would result in the overall plant to be controlled. Interestingly, the asymmetrical mutex specification described above cannot be depicted as a finite state automaton; the best approximation that can be accommodated allows some finite maximum of simultaneous readers. Both of the foregoing restrictions will be circumvented in the proposed solution.

<sup>2</sup> A string  $p$  is a prefix of another string  $s$  if there is a string  $x$  such that  $s = px$ . The notation  $\overline{\mathcal{L}}$  denotes the *prefix closure* of the language  $\mathcal{L}$ : the language of all strings which are prefixes of strings in  $\mathcal{L}$ . A language with the property  $\mathcal{L} = \overline{\mathcal{L}}$  is called *prefix-closed*.

<sup>3</sup> It may be, of course, that the resultant language is empty.

```

public class R extends Thread {
    public void read() {};
    public void run() {
        while(choiceContinue()) {
            read();
        }
    }
}

public class W extends Thread {
    public void write() {};
    public void run() {
        while(choiceContinue()) {
            write();
        }
    }
}

```

(a) R. (b) W.

Fig. 1. The Java code for the Reader (R), Writer (W) process objects.

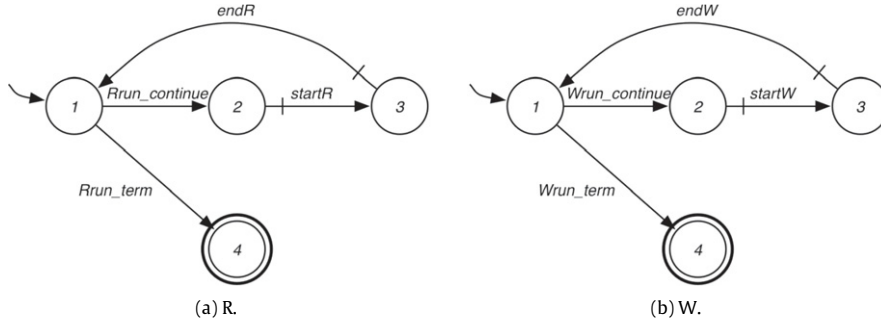


Fig. 2. The FSA representations of the R and W objects' behavior.

### 2.1. Limited lookahead

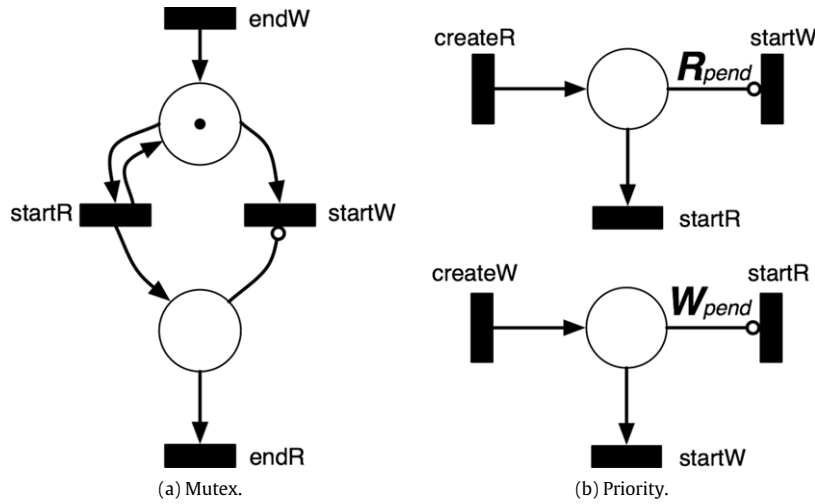
The notion of limited lookahead for discrete-event systems was first explored by Chung, Lafortune and Lin [4] in order to find ways to mitigate the large state-space search cost of computing the plant. It has as its objective, the computation of an effective control decision based on exploration of only a finite neighborhood of the present state. The dynamic DES paradigm of Grigorov and Rudie [20] extends the limited-lookahead idea to encompass plants which change over time. The future plant behavior, in addition to being conceivably too large to compute at once, is also unknown.

Central to the concept of limited lookahead is the definition of a *window* of depth  $N$ , up to which exploration is carried out. The window is a graph comprised of states resulting from possible event sequences up to length  $N$ : in effect a neighborhood of the current state. The edge of that neighborhood, called the *lookahead depth*, partitions the languages  $\mathcal{L}(S/G)$  and  $\mathcal{L}_m(S/G)$  of the system into sublanguages based on the length  $N$ . Beyond that length, the controllability and legality of the languages are uncertain; this uncertainty is dealt with by means of a *lookahead policy*. The generated control results in disablement of all controllable events deemed contrary to the lookahead policy.

Chung et al. contemplated a spectrum of policies, the extremes of which they investigated in their paper: a *conservative* policy regarding all pending traces as illegal, and an *optimistic* policy regarding all pending traces as both legal and nonblocking. When a pending trace is treated as legal and nonblocking by the policy, it is possible for new information to arise showing the path as illegal or blocking once it has been followed; Chung et al. define this as a *runtime error* and show that in a system with a conservative lookahead policy, if a runtime error does not occur in the initial state, it will never occur in the execution [4].

Where the work of Chung et al. focuses on retaining in the limited lookahead framework the maximum number of provable properties from standard DES, Grigorov and Rudie [20] generalize the static DES paradigm for the case of a temporally mutable plant—one which can change between event firings. Since the plant can change at any time, any guarantees regarding nonblocking and permissiveness properties are no longer relevant, and the purpose of control moves from strict prevention of constraint violation to avoidance.

This shift in attitude toward supervisory control is evident in two major formalism changes: first, in the discarding of the marked states concept, which Grigorov and Rudie replace with a value function  $v(s)$  mapping each state to a utility (which also considers the probability of blocking); second, representation of plant mutability through the appearance and disappearance of plant *modules*—whose structure and behavior are not necessarily known prior to runtime—at each step in the event sequence. Since the plant neighborhood generated by limited lookahead represents a forecast based only on the current plant, it has an inherent limitation in its reliability with respect to future plant changes. Given a probability of plant change, Grigorov and Rudie propose a definition of plant reliability—intuitively, a measure of how long one can expect the plant to remain stable—and normalize their value function for future event sequences to account for the probability such event sequences will actually be accessible. This function is in effect a limited lookahead policy, based on a criterion other than simply incidence of the lookahead boundary. Such a policy will be shown not to be necessary in this work's treatment of limited lookahead due to a differing treatment of plant mutability based on Petri nets (see Section 2.2). Finally, Grigorov



**Fig. 3.** Mutual exclusion (mutex) and priority specifications for the Readers/Writers problem.  $R_{pend}$  and  $W_{pend}$  are integer “weights” which describe the multiplicity of arcs connecting the respective places/transitions in each extended Petri net.

and Rudie propose a limited-lookahead search algorithm based on an  $A^*$ /game tree search in which controllable events are treated as protagonist moves, the values of which are to be maximized, and uncontrollable events as antagonist moves, the values of which are to be minimized. They further show this search method to be provably as safe as the optimistic search of Chung et al., and through the use of a monotonically overoptimistic search heuristic, guarantee that the search is optimal with respect to its value function.

## 2.2. Petri nets

Many variants on Petri nets have been described in the literature [5], but the broadest definition is simply as a four-tuple:

$$N = (P, T, I, O)$$

where

- $P$  is the set of *places*;
- $T$  is the set of *transitions*;
- $I : T \rightarrow 2^P$  is a set of *inputs* from places to transitions; and
- $O : T \rightarrow 2^P$  is a set of *outputs* from transitions to places.

The above definition is amenable to representation as a bipartite directed graph with places and transitions as nodes. An arc from a place to a transition specifies an input to the transition; an arc from a transition to a place specifies an output from that transition. Successive application of the transition function defines a *reachability graph*—a (possibly infinite) state automaton—which is constructed recursively for each generable event at a given marking.

In a *marked Petri net*, each place can be *marked* with an integral number of *tokens*; the overall state of the net is specified by the distribution of tokens in places, called the *marking* of the Petri net. If the places are indexed, the marking can be represented as a vector of nonnegative integers. A transition may *fire* if every input place has at least one token; *firing* entails the removal of a token from each input place for every arc from that place to the transition, and addition of a token to each output place for every arc from the transition to that place.

An *extended Petri net* results from the addition of a second type of arc, called an *inhibitor arc*, which also connects a place with a transition. In an extended Petri net, a transition  $t$  fires only when all of  $t$ 's input places have at least one token and all inhibitor arcs ending at  $t$  have no tokens, i.e., are empty. Inhibitor arcs thus allow for “zero-testing” of a place. In Fig. 3(a), for instance, the firing of **startR** disables transition **startW** until **endR** fires and the place that is connected to **startW** via the inhibitor arc (shown with a small circle at the end) becomes empty.

Extended Petri nets have the expressive power of Turing machines [5], with all the attendant decidability characteristics entailed: any code-able constraint specification is expressible, but issues such as reachability and liveness move from being merely intractable<sup>4</sup> to undecidable in theory. In the opposite extreme, any bounded Petri net only has the expressive power of regular languages; examples are *state machine Petri nets*, i.e., Petri nets with only one input and one output per transition.

Fig. 3 gives the specifications desired in the Readers/Writers problem: both the aforementioned asymmetrical mutex, and a specification illustrating the notion of *priority*: if a certain number of Readers are currently queued while waiting for writers to terminate, then further creation of Writers will be suspended until at least one Reader is allowed to execute—

<sup>4</sup> While both reachability and liveness have been shown to be decidable in a standard (non-extended) marked Petri net, they are undecidable for nets with inhibitor arcs [21,22].





```

public class ThreadA extends Thread {

    public void run() {
        if(condition)
            // sup.event: alpha
            executeAlpha();
        else
            // sup.event: beta
            executeBeta();
    }
}

```

Fig. 5. Thread with two controllable paths.

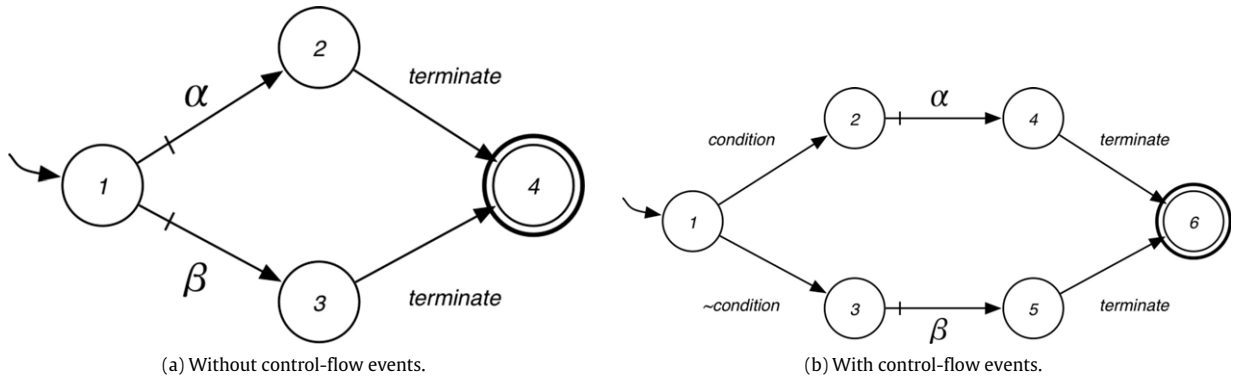


Fig. 6. An automaton representation of Thread A with and without explicit representation of control-flow events indicating branching.

As asserted above, the current state can be denoted with a vector, each element giving a thread count for each of the possible states of the underlying automata. For example, if the Petri net's places are numbered as in Fig. 4, then the vector

$$[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]$$

denotes the initial marking of the plant; namely, one RW thread in its start state, and no other threads present.

### 3.1. Control flow and controllability

The proposed supervisor exerts control upon the target code in only one way: by instructing one or more threads to suspend execution until a condition is met. This action is called *blocking*<sup>5</sup> the thread, and a thread in this state is also said to be *blocking*. The actual control flow of the thread is not exposed to control by the supervisor; once a particular execution sequence is chosen by the thread, that sequence must eventually occur. The supervisor controls the overall execution sequence of the plant merely by postponing the individual threads' sequences in order to realize a global outcome.

In discrete-event systems, the controllability property fits this modeling artifact exactly. During code instrumentation, events are defined for every entry and exit to a control flow block, and for code points which are considered specification-relevant: that is, points that have the potential to cause the specifications to be violated. The former shall be termed *control-flow events* and the latter *specification events*, or *specification-relevant events*.<sup>6</sup> A critical design decision for supervisory control is the mapping of these two classes of events to the controllability partitioning of the prospective model's alphabet; control-flow events are designated uncontrollable and specification events controllable.

The following example outlines the motivation underlying the foregoing design decision by showing an example of execution that can result in blocking that is undetectable by a supervisor which does not explicitly model control-flow events. Consider a thread, Java code for which is provided in Fig. 5.

Under transformation to an automaton, this thread has two specification events  $\alpha$  and  $\beta$ , and two control-flow branches. There are three control-flow events (one for each possible branch, and one for thread termination). Fig. 6 gives two possible automata for this code under transformation: while 6(a) shows the result of a transformation simply representing the two

<sup>5</sup> In an unfortunate collision of terminology from the software engineering and discrete-event systems worlds, the term *blocking* has performed two meanings in this work: in the former sense, simply the suspension of a thread, and in the latter the equivalent of live-lock—the inability to reach a terminal state.

<sup>6</sup> Since the overall objective of our work is to use Discrete-Event Systems for the generation of concurrency control, the specification is expected to describe how the execution of the concurrent processes should be interleaved; the specification events can thus also be thought of as *concurrency events*.

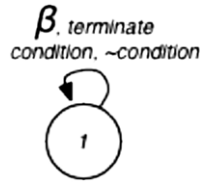
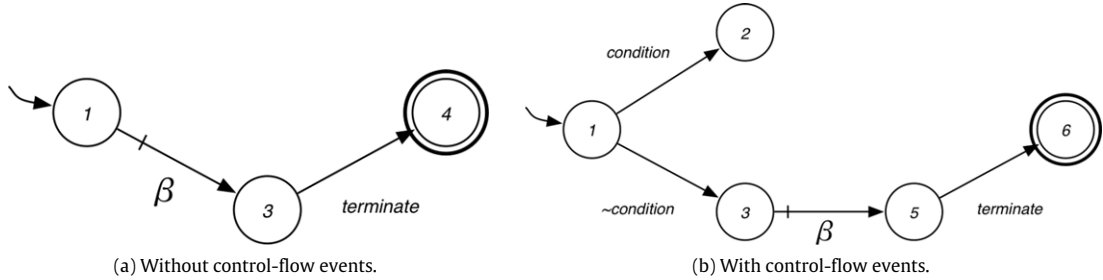
Fig. 7. An automaton specification disabling  $\alpha$ .

Fig. 8. Automaton representations of Fig. 6 under the control of the specification depicted in Fig. 7.

possible concurrency outcomes, 6(b) explicitly represents the control flow choice made by the thread independent of the specification event actually occurring.

Next, a potential specification of constraints is applied to this thread. Fig. 7 gives a very simple automaton which represents universal disablement of  $\alpha$  (while allowing all other plant events), and Fig. 8 shows the subset of plant behavior that is legal with respect to this specification.

Finally, we speculate that, the value of the Boolean condition is **true**. In the case of Fig. 8(a), it is clear that the only path allowable by the supervisor is  $\langle \beta \text{ terminate} \rangle$ ; however, the code itself will choose the path including the firing of event  $\alpha$ , and then block until  $\alpha$  is subsequently enabled. As that cannot occur under this specification, blocking has occurred which is not accounted for in the model; thus code under its control may not be deadlock-free.

The blame for this discrepancy lies in the fact that the DES model of the system does not incorporate runtime information: the value of condition is unavailable to the model and hence to the supervisor. Fig. 8(b) gives the solution: the value of condition is built into the structure of the model, and now a supervisory decision can be made that could entail preventing the system from even entering state 1.<sup>7</sup>

In summary, the scenario omitting explicit control flow mapping admits deadlocked states not detectable by the supervisor; under the adopted paradigm—mapping control flow to uncontrollable events and specification-critical points to controllable ones—this class of deadlocks is taken into account by the model.

Returning to Fig. 4, uncontrollable events are evident for such junction points in the code as the evaluation of the condition of an **if** statement or **while** loop, such as **Rrun\_loopT** denoting the entry or re-entry to the while loop in the reader thread.

### 3.2. Marked states and terminability

The limited lookahead paradigms discussed in Section 2.1 both lack information with respect to the terminability of a program—namely, that a program will eventually reach a state in which it is valid to halt. A new criterion for blocking is required, therefore, that acts conservatively with respect to control policy (i.e., still guarantees the nonblocking property) but can enact control decisions based only on the information provided within the specified lookahead depth.

To that end, we redefine the term “nonblocking state” to mean a state from which any disabled event is eventually enabled within the lookahead window along any path in the subtree of that state. This is not only determinable by definition within the lookahead depth, but maintains the blocking guarantee (with a certain qualification to be described in Section 3.4) and can be performed in only the time required for one tree traversal—that is, no extra time is required above that for the state illegality check. We refer to this re-enablement property of an event  $e$  by saying that,  $e$  is *supported* if whenever  $e$  is disabled in some state  $s$  it becomes enabled within the lookahead window along any path from  $s$ .

It is instructive to refer to the controllability partitioning described in Section 3.1. As a result of control flow being explicitly mapped to uncontrollable events, it is apparent that any thread has at most one controllable event per state: multiple specification-relevant code points that are part of the same control-flow block (each necessarily annotated with a separate controllable event) imply a control-flow decision made prior to the execution of whichever point is ultimately

<sup>7</sup> Of course, if this automaton constitutes the whole plant, then the initial state itself is illegal, leading to a runtime error. In this case, it is said that the specification is invalid with respect to the plant. Were this automaton to constitute a subgraph of the plant, then the subgraph would be avoided entirely as the **supcon** calculation would eliminate state 1 during exploration of its predecessors.



traversed. This control-flow decision would be annotated with a set of alternative uncontrollable events, separating the controllable ones into different states. Thus a controllable event that is disabled but generated by the plant indicates the existence of a blocked thread. To prevent a system deadlock (the permanent blocking of one or more threads), the event must be re-enabled at some time in the future.

If the above criterion of re-enablement is met for every event, then no thread will remain deadlocked in perpetuity; therefore, the system is nonblocking (again, subject to an important proviso to be outlined in Section 3.4). Clearly the criterion is a very conservative one; it may be that an event that remains disabled for a future event sequence of length  $N$  may be re-enabled at step  $N + 1$ , so the system is not necessarily maximally permissive.

### 3.3. Preliminary definitions

So motivated to redefine the notion of a nonblocking state, we derive a new nonblocking language as per the lookahead function outlined by Chung et al. (Section 2.1) which uses the notion of event support, rather than depending on marked states.

The new language can be described with the aid of some extra notation (the first three definitions taken from [4]):

**Definition 3.1.** The *postlanguage* of a string  $s \in \mathcal{L}$  is the set of strings in  $\mathcal{L}$  with the prefix  $s$ :

$$\mathcal{L}/s = \{t \in \Sigma^* \mid st \in \mathcal{L}\}.$$

**Definition 3.2.** The *right quotient language* of  $\mathcal{L}$  with respect to a language  $K$  is the set of all prefixes  $s$  that can be extended by a string in  $K$  to a string in  $\mathcal{L}$ :

$$\mathcal{L}/K = \{s \in \Sigma^* \mid (\exists t \in K \mid st \in \mathcal{L})\}.$$

**Definition 3.3.** The *truncation* of  $\mathcal{L}$  to depth  $N$  is the set of all strings in  $\mathcal{L}$  of length  $N$  or less:

$$\mathcal{L}|_N = \{s \in \mathcal{L} \mid |s| \leq N\}.$$

**Definition 3.4.** A string  $s \in \mathcal{L}$  is said to *support* an event (or symbol)  $\alpha \in \Sigma$ , written  $s \sqsupset \alpha$ , if there are strings  $x$  and  $y$  in  $\Sigma^*$  whose sequential concatenation with  $\alpha$  yields  $s$ , i.e.,  $s = x\alpha y$ , and for a given  $\alpha \in \Sigma$  the  $\alpha$ -*support* of a language  $\mathcal{L}$  is the set of all strings in  $\mathcal{L}$  containing  $\alpha$ :

$$\mathcal{L}^\alpha = \{s \in \mathcal{L} \mid s \sqsupset \alpha\}.$$

Since all traces leading to a state share a single postlanguage, we avoid introducing explicit notation for states by referring only to these postlanguages. This gives our desired formal definition of support of an event in (or at) a given state:

**Definition 3.5.**  $\alpha$  is said to be *supported to length  $N$  in state  $q = \hat{\delta}(q_0, s_0)$*  if the intersection with  $\mathcal{L}/s_0$  of the truncation to  $N$  of the  $\alpha$ -support is nonempty, that is, a string  $s$  with  $|s| < N$  that supports  $\alpha$  exists in  $\mathcal{L}/s_0$ .

The notion of support to a specific length is mandated by the limited nature of our lookahead; it represents a conservative approach to the determination of support manifest in the assumption that if support is not found within  $N$  steps, it will not be found. This allows us to avoid reaching states without support for some event at possibly the cost of eliminating fully supporting states (states that support all events).

The intuition of the above discussion relating software threads to event sequences is that, no controllable event be disabled in perpetuity; hence *all* controllable events must have support in any reachable state. Restricting ourselves to a finite window as above we arrive at the definition:

**Definition 3.6.** The state  $q = \hat{\delta}(q_0, s_0)$  can be said to be *nonblocking to length  $N$*  if

$$\bigcup_{\alpha \in \Sigma_c} (\mathcal{L}/s_0)^\alpha|_N \supseteq \mathcal{L}/s_0$$

that is, every sequence in  $\mathcal{L}/s_0$  supports all controllable events.

The algorithm implementation that achieves this is described in Section 4. A brief and simple example of the intuition behind the desired algorithm follows; thereafter, two important considerations are noted in the following Sections 3.4 and 3.5 before turning to the problem of formalizing the definitions of nonblocking and support to meet the goal outlined above (Section 4).

In Fig. 9, a section of a lookahead tree is shown for a system with an alphabet of (at least) three events  $\alpha$ ,  $\beta$  and  $\gamma$ , with  $\{\alpha, \gamma\} \subseteq \Sigma_c$  and  $\beta \in \Sigma_u$ . In the diagram, a dashed-line notation is used to indicate disablement (by implication of an unshown specification). At a state  $q$ ,  $\alpha$  is disabled; this implies the existence of a software thread which is prevented from continuing (by executing  $\alpha$ ). If  $\alpha$  is never re-enabled, the thread will remain in its current state, unable to come to a normal termination.

The sequence  $\gamma\gamma$  causes  $\alpha$  to be re-enabled; since this path is legal from  $q$ ,  $\alpha$  is supported at  $q$  to length 2. However, due to the uncontrollable  $\beta$  event at  $\delta(q, \gamma)$ , it is possible for the system to be taken uncontrollably to a state where  $\alpha$  is again disabled, invalidating the support derived from event sequences available at  $\delta(q, \gamma)$ . Therefore state  $q$  is not *nonblocking*.

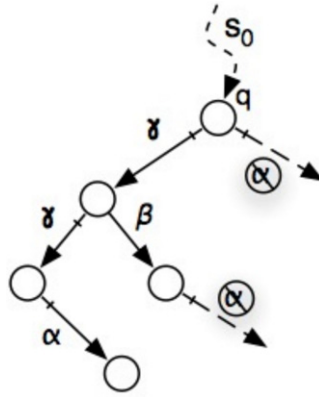


Fig. 9. An example of support and nonblocking properties—the state  $q$  is not nonblocking.

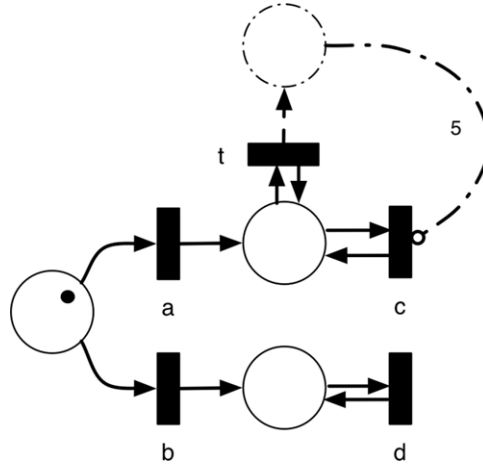


Fig. 10. An extended Petri net system whose specification disables  $c$  after five firings of  $t$ . The number 5 annotating the inhibitor arc indicates the five-token requirement.

### 3.4. Lookahead length

Determining the lookahead length is heavily problem-dependent. The two main factors in determining the appropriate length are system resources and blocking characteristics of the plant.

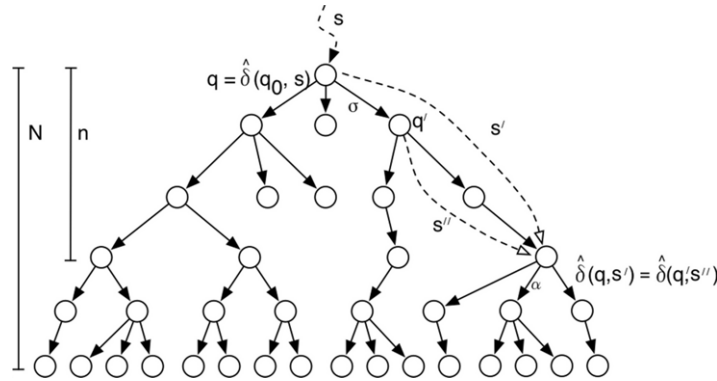
The amount of system resources used is dependent upon the number of nodes generated at a given lookahead depth. The ability to estimate the lookahead length to which the state space can be economically explored, is an important avenue of future work (see Section 9.1). Winacott et al. [23] have explored ways to quantify the system resources used by lookahead at a given depth.

As the lookahead algorithm is conservative with respect to event disablement, if an event is disabled it is assumed to preclude being nonblocking unless it is shown to be re-enabled in a future state. The algorithm enforces the nonblocking property to the extent of the lookahead windows; however, the algorithm is not guaranteed to be nonblocking in the traditional DES sense.

To illustrate this, we consider the following small example of a plant/specification Petri net, depicted by Fig. 10. To show the entire system, the plant and specification are included in a single diagram; the dashed lines indicate the specification.

It is clear that the plant initially has two generable choices:  $a$  or  $b$ . Were  $a$  to be executed,  $t$  would become generable. With five  $t$  firings,  $c$  would be permanently disabled; to prevent this,  $t$  would eventually have to be permanently disabled. Assuming that transitions  $t$  and  $c$  are mapped to two threads, either case leads to permanent blocking of at least one thread: the one waiting on the disabled event. As a result, blocking is inevitable if  $a$  is generated at the outset; this is not apparent, however, until at least six event firings later. We call this scenario a *blocking well*: a subtree of the state space of the Petri net which, once entered, guarantees blocking regardless of the future event sequence. A blocking well has an associated depth (the *blocking well depth*); this is the length of the shortest sequence leading to a blocking state. In the example of Fig. 10 the depth of the blocking well is six.

If the lookahead length is shorter than the blocking well depth, the lookahead algorithm will allow entry to the blocking well. At some point after further event firings, the lookahead will fail to find a safe execution path and will terminate with a runtime error when the current state is marked blocking.



**Fig. 11.** Lookahead exploration starting from a node in state  $q = \hat{\delta}(q_0, s)$  after a prior trace  $s$ . The node currently being explored has state  $\hat{\delta}(q, s') = \hat{\delta}(q, s'') = \hat{\delta}(q', s'')$ .

Conversely, as long as the lookahead length is longer than the depth of any blocking well in the system, it will be guaranteed to produce a nonblocking result (though maximal permissiveness is not guaranteed). The computational complexity of determining whether a blocking well exists in the reachability graph of a given Petri net is not known; in particular, we do not know if the time to determine whether a blocking well exists is any shorter than the time required to compute the reachability graph itself. Further investigation is required of possible restrictions under which blocking wells can be computed.

### 3.5. Specification expression

Specifications can be expressed in any way that can be mapped to a Boolean function evaluating event legality in a given state. In this work, attention has primarily been given to using Petri nets due to their expressive power, as well as to maintain a uniform conceptual framework when reasoning about the plant and specifications together. It is trivial to compose a complete system of plant and specifications, merely by lining up transitions labeled with the same event in the individual modules, as long as transitions within each module are uniquely labeled with events, and with only one event per transition. General Petri nets do not have these two properties; nonetheless, the latter is a direct result of our mapping of supervisor calls to transitions and to the fact that each call takes exactly one argument (that being the event label), and the former, we are at liberty to enforce in our choice of the value of that argument.

As mentioned, the use of extended Petri nets gives the specification formalism the expressiveness of Turing machines. This in particular motivates the use of limited lookahead, as determining the legality of an arbitrary event sequence is equivalent to determining the membership of a given arbitrary string in the legal language  $E$ ; this is not possible in general for Turing machine languages. A limited lookahead window imposes a sequence length after which it “gives up”, effectively working around the issue of halting and thus the possibility of nontermination.

## 4. Supervisor lookahead algorithm

Before describing the lookahead traversal algorithm it is necessary to describe the space it traverses; namely, the closed-loop system’s overall state space. The plant is at any time in a specific state embodied by the distribution vector of tokens in the plant’s Petri net. Each individual specification indicates event legality by tracking an internal state as well, regardless of its representation (finite state automaton, Petri net, etc.). The model  $M$  thus has a well-defined state composed of the aggregate state of all the underlying components; as the lookahead is designed to be indifferent to these components’ implementations, it is this model state to which we shall hereafter refer. To distinguish between the states of the underlying automata and the overall lookahead, the term “node” shall refer to the nodes of the lookahead tree, in which the states of  $M$  are thought to be embedded.

If  $N$  is the length of the lookahead window, then the blocking property is unknown beyond it; thus without loss of precision, we will use the term “nonblocking” to mean “nonblocking to length  $N$ ”, where it is clear that, the context includes a lookahead length. This definition of nonblocking yields the language desired from Section 3.2, as shown by the following contemplation of a lookahead tree, starting at a node in some state  $q = \hat{\delta}(q_0, s)$ , encountering a state  $\hat{\delta}(q, s') = \hat{\delta}(q_0, ss')$  after exploring a further trace  $s'$  in the postlanguage  $\mathcal{L}(G)/s$ . This scenario is depicted with a subset of the lookahead tree in Fig. 11.

We now turn to the construction of the lookahead algorithm by defining an appropriate tree-traversal recursion. To do so, we reformulate the definitions of support and nonblocking in a way that is amenable to recursive expression. We first state an inference that the foregoing definition of support given in Definition 3.5 is equivalent to the following:

**Corollary 4.1.** For a language  $\mathcal{L}$  and state  $q = \hat{\delta}(q_0, s)$  with an event  $\alpha \in \Sigma_c$ ,  $\alpha$  is supported in  $q$  to length  $N$  iff:

1.  $\alpha$  is legal in  $q$  for  $N \geq 0$ , or
2.  $\alpha$  is supported in  $\delta(q, \sigma)$  to length  $N - 1$  for some  $\sigma \in \Sigma$ ,  $N > 0$ .

**Proof.** Suppose  $\alpha$  is supported in  $q = \hat{\delta}(q_0, s)$  to length  $N$ . Then there is some string  $s'$  such that  $|s'| < N$  and such that  $\alpha \in \mathcal{L}/ss'$ ; that is,  $\alpha$  is legal at  $\hat{\delta}(q, s')$ . Note that,  $s'$  could be the empty string  $\epsilon$ , trivially satisfying 4.1 through legality; if it is not, then since  $s' = \sigma s''$  for some  $\sigma \in \Sigma$  and  $s'' \in \Sigma^*$  and  $\alpha$  is supported to  $N$  at  $q$  by  $\sigma s''$ , it is supported to  $N - 1$  at  $q' = \delta(q, \sigma)$  by  $s''$ , which has length  $N - 1$ . Thus support implies (1) and (2).

Now suppose  $\alpha$  is legal in state  $q = \hat{\delta}(q_0, s)$ ; then  $\alpha$  is supported trivially since  $\alpha$  is (“re”-)enabled under the empty string. If  $\alpha$  is not legal at  $q$  but is supported to length  $N - 1$  at  $\delta(q, \sigma)$  for some  $\sigma$ , then there is some  $s'' \in \Sigma^*$  such that  $\hat{\delta}(\delta(q, \sigma), s'')$  supports  $\alpha$ . But  $\hat{\delta}(\delta(q, \sigma), s'') = \hat{\delta}(q, \sigma s'')$ , so  $\alpha$  is supported in  $q$ . Thus (1) and (2) imply support, and equivalency is established.  $\square$

We now state a recursive reformulation of Definition 3.6 (nonblocking) and show equivalency.

**Corollary 4.2.**  $q$  is nonblocking to length  $N$  iff:

1.  $\forall \alpha \in \Sigma_c$ ,  $\alpha$  is supported to length  $N$ , and
2. either  $N = 0$  or  $\forall \beta \in \Sigma_u$ ,  $\delta(q, \beta)$  is nonblocking to length  $N - 1$ ,  $N > 0$ .

**Proof.** Suppose  $q = \hat{\delta}(q_0, s)$  is nonblocking to length  $N$ . Then all controllable  $\alpha$  are supported to length  $N$  directly by Definition 3.6. If  $N = 0$  then certainly no strings are included which do not support  $N$ ; if  $N > 0$ , then suppose there is  $\beta \in \Sigma_u$  such that  $\delta(q, \beta)$  is not nonblocking to length  $N - 1$ . Then since  $\delta(q, \beta) = \hat{\delta}(q_0, s\beta)$ ,  $\hat{\delta}(q_0, s\beta)$  is not nonblocking to length  $N - 1$ , which means there is a string  $s' \in \mathcal{L}/s\beta$  that does not support to  $N - 1$  some  $\gamma \in \Sigma_c$ . But then  $\beta s'$  does not support  $\gamma$  to  $N$  but is clearly in  $\mathcal{L}/s$ , contradicting Definition 3.6 (2). Thus nonblocking implies (1) and (2).

Now suppose that,  $q = \hat{\delta}(q_0, s)$  is not nonblocking to length  $N$ . This means the postlanguage after  $s$  contains a sequence  $s'$  not supporting some  $\alpha \in \Sigma_c$  to length  $N$ . This is trivially accommodated if  $\alpha$  is not directly supported at  $q$  to length  $N$ ; if it is,  $N$  must be greater than zero since  $N = 0$  and  $q$  not nonblocking implies some  $\alpha$  must be disabled at  $q$ , contradicting the assertion that  $\alpha$  is supported. Given  $N > 0$  and support for  $\alpha$  at  $q$ , we attempt to characterize  $s'$ .

Firstly, let  $s' = \beta s''$  with  $\beta \in \Sigma$  and  $s'' \in \Sigma^*$ . Since  $s'$  does not support  $\alpha$  to  $N$ ,  $s''$  does not support  $\alpha$  to  $N - 1$ ; thus  $\delta(q, \beta)$  must not be nonblocking. If  $\beta \in \Sigma_c$  then  $\beta$  would be disabled, implying  $s''$  is not in the postlanguage  $\mathcal{L}/s\beta$ ; but this would imply that  $s'$  is not in the postlanguage  $\mathcal{L}/s$ , contradicting the hypothesis. Thus  $\beta \in \Sigma_u$ . This shows that (1) and (2) imply nonblocking, and concludes the proof of equivalency.  $\square$

Turning our attention to the illustrative lookahead tree depicted in Fig. 11, we apply the above from  $q$  and distinguish two cases:

- If  $q = \hat{\delta}(q_0, s)$  is nonblocking under Definition 4.2, then for every controllable event  $\alpha \in \Sigma_c$  the support property holds, meaning that one of the following is true:
  1.  $\alpha$  is legal at  $\hat{\delta}(q_0, s)$ , implying that  $s\alpha \in E|^\alpha|_N$ ;
  2.  $\alpha$  is legal at  $\hat{\delta}(q_0, ss')$  for some  $s' \in \mathcal{L}(G)/s|_N$ . This means that  $ss'$  is legal, so  $s$  is a prefix of a string in  $E|^\alpha|_N$ .
- If  $q = \hat{\delta}(q_0, s)$  is not nonblocking under Definition 4.2, then one of its generable controllable events  $\alpha$  is not supported, meaning that, it is disabled at  $q$  and not re-enabled before the end of the lookahead—at every node of the subtree of  $q = \hat{\delta}(q_0, s)$  of depth  $N$ ; this means  $s$  is not a prefix of  $E|^\alpha|_N$ .

The algorithm will assign a value of `true` or `false` to the nonblocking property of each node  $q$  (at depth  $n$ ) according to the following criteria:

1. for all events  $\alpha \in \Sigma_c$  generable at  $q$ , there exists a node  $q'$  reachable from  $q$  in  $N - n$  steps, such that  $\alpha$  is legal at  $q'$ , and
2. for all generable events  $\alpha \in \Sigma_u$ , all nodes  $\delta(q, \alpha)$  are nonblocking.

Fig. 12 gives the lookahead pseudocode, a recursive traversal of the states generated by the plant/specification system, checking the properties defined by Corollaries 4.1 and 4.2. The inputs to the algorithm are the current state  $q$ , the current search depth  $n$ , the plant language  $\mathcal{L}(G)$  and the legal language  $E$ .

The algorithm recursively explores the tree by firing events (lines 8–14) up to the recursion depth  $N$ ; all controllable events are marked blocking in the leaves (lines 2–6). On returning from the recursion (line 15), each path to a leaf must traverse a node for every controllable event in which that event is legal (lines 16–23); otherwise the event is marked blocking in the state embedded in the node, and thus the node is marked blocking. In addition, nodes with uncontrollable events leading to blocking nodes will be marked blocking (lines 27–29).

## 5. Online supervisor synthesis

The objective of the supervisor is to use the software model and constraints to effect control decisions at runtime. The process followed in creating the dynamic supervisor is largely analogous to that of Dragert et al. [1] with the following caveats. In the static case, the system (i.e., the model of the plant under control) is simply converted into a control code (see Section 2) which is consulted as events are fired by the plant. In the dynamic case, this construct is replaced by an online limited-lookahead search. The supervisor retains separate models for each system component: a composite Petri net for the plant and separate specifications in the chosen modeling formalism. At runtime, these are traversed in lockstep by the lookahead algorithm to determine the eligibility of each event to fire.

```

1  function lookahead( $q, n, \mathcal{L}(G), E$ )
2      if ( $n = 0$ )
3          for each  $\alpha \in \Sigma_c$ 
4               $q.\text{eventIsSupported}[\alpha] = \text{false}$ 
5               $q.\text{isNonBlocking} = \text{false}$ 
6          return
7      else
8          for each  $\alpha \in \Sigma$ 
9              if  $\delta(q, \alpha) \in E$  //  $\alpha$  is legal in state  $q$ 
10                 lookahead( $\hat{\delta}(q, \alpha), n - 1$ )
11              $q.\text{isNonBlocking} = \text{true}$ 
12             for each  $\alpha \in \Sigma_c$ 
13                 if  $\delta(q, \alpha) \in E$  //  $\alpha$  is legal in state  $q$ 
14                      $q.\text{eventIsSupported}(\alpha) = \text{true}$ 
15                 else
16                      $q.\text{eventIsSupported}(\alpha) = \text{false}$ 
17                     for each  $\alpha' \in \Sigma - \{\alpha\}$ 
18                         if  $\delta(\hat{\delta}(q, \alpha'), \alpha) \in E$  //  $\alpha$  is legal in state  $\hat{\delta}(q, \alpha')$ 
19                              $q.\text{eventIsSupported}(\alpha) = \text{true}$ 
20                     if NOT  $q.\text{eventIsSupported}(\alpha)$ 
21                          $q.\text{isNonBlocking} = \text{false}$ 
22             for each  $\alpha \in \Sigma_u$ 
23                 if not  $\hat{\delta}(q, \alpha).\text{isNonBlocking}$ 
24                      $q.\text{isNonBlocking} = \text{false}$ 
25     return

```

Fig. 12. Pseudocode for lookahead algorithm.

The methodology is described in terms of a pipeline process. The inputs of the process are:

1. the source code of the software to be controlled—the *raw source*;
2. knowledge of the points in the code which are specification-relevant—the *event information*;
3. the *constraints* to be imposed on the software.

The process is composed of:

1. Source code markup;
2. Thread model generation;
3. Constraint specification;
4. Supervisor model generation;
5. Supervisor code translation;
6. Supervisor call insertion.

The flow chart of Fig. 13 depicts the process and indicates the inputs and outputs of each step of the enhanced methodology based on dynamic lookahead. The steps are individually treated in the subsequent subsections, one subsection for each step. The focus is on highlighting the differences between dynamic lookahead-driven control and the static case.

### 5.1. Source code markup

This phase is largely the same as that of Dragert et al. [1]: the raw source code is taken as input by a developer or designer and annotated to indicate the specification-relevant points.<sup>8</sup> The controllability/control-flow partitioning allows for the omission of markup for control flow events altogether; the developer need only identify and add annotations for specification relevance, as the next step produces uncontrollable events for all control flow as part of its process.

The source code for the *R* and *W* threads of our Readers/Writers example of Section 2 (see Fig. 1) is depicted in Fig. 14, with markup indicating the controllable, specification-relevant events.

### 5.2. Thread model generation

From the markup, a control-flow graph is generated representing the possible sequences of code instructions. The result is a collection of finite state automata  $\{G_i\}$  in a machine-readable format: one for each thread in the software. A control-flow graph with a separate transition for every code step is not necessary; only control-flow boundaries, such as entry/exit points to *for*, *while* and *if* blocks need be represented by event transitions, in addition to the explicit concurrency points marked up in the first step (the former uncontrollable, the latter controllable).

<sup>8</sup> Note that such a point is located not on a particular line of code, but between two lines of code.

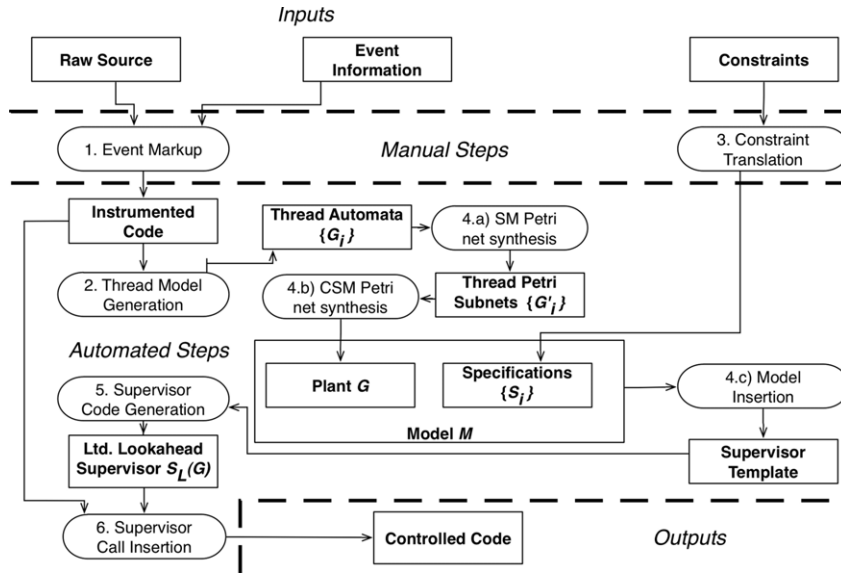


Fig. 13. Flow chart depicting the Supervisor autogeneration process.

```

public class R extends Thread {
    public void read() {};
    public void run() {
        while(choiceContinue()) {
            // event: startR
            read();
            // event: endR
        }
    }
}
(a) R.

public class W extends Thread {
    public void write() {};
    public void run() {
        while(choiceContinue()) {
            // event: startW
            write();
            // event: endW
        }
    }
}
(b) W.

```

Fig. 14. The Java code for the R and W objects marked up with concurrency-relevant events (indicated by // event lines).

The control-flow graph generation step can be automated. Generation of a control-flow graph from code is a well-known technique, already used in [1] and other discrete-event control methods [11]. The implementation contemplated for this methodology is based on the JavaCC parser generator; example grammars already exist for parsing general Java code, with the ability to add inline Java to output arbitrary translations. The idea is to parse out control-flow-related blocks and produce XML in the format read by the IDES discrete-event systems analysis tool [16]. From IDES, the developer can create specifications and reason about concurrency using the DES representation of the plant. The individual thread models are then re-importable to the supervisor code to inform the structure of the lookahead.

### 5.3. Constraint specification

Next, the constraints must be translated from high-level assertions (either in formal logic or plain language) to a collection  $\{S_i\}$  of specifications in a modeling domain, upon which parallel composition with the plant can be performed. As discussed in Section 3, there is no requirement to use finite state automata as the vehicle for expressing the specifications; indeed, their use limits the expressiveness of the constraints to regular languages. Any formalism meeting the parallel composition criterion above can be used; in the work described herein specification expression is accomplished with extended Petri nets. No explicit translation from other codifications, such as temporal logic, has been explored; it has been left to the software designer to synthesize appropriate Petri nets that capture the global constraint required. The use of Petri nets for the specification of concurrent systems appears to be somewhat intuitive, as it explicitly represents the “flow” of multiple objects through a set of states.

The modeling of specifications as Petri nets is functionality not yet available in IDES; however, with the addition of such functionality, along with the control-flow graph generation, an end-to-end automated process would be available to developers for online-lookahead-based control code generation, as the insertion of supervisor code—in this case the lookahead algorithm with imported finite state automaton and Petri net models—is already automated for Java in the proof-of-concept implementation associated with this work.



Fig. 3 shows the specifications relevant to our example.

#### 5.4. Supervisor autogeneration

It is in this step that, the process for the dynamic case significantly diverges from the static case. Rather than a series of compositions resulting in a single automaton, the plant and specifications are embedded in a *model*  $M$  which can be used to dynamically explore the state space of the system.

As with the static case, the step is broken up into three sub-steps:

##### (a) SM Petri net synthesis:

For each thread automaton  $G_i$ , a state machine (SM) Petri net  $G'_i$  is generated. First, a one-to-one mapping of states from  $G_i$  to places in  $G'_i$  is defined. Next, for each state  $q$  in  $G_i$ , the outgoing arcs are enumerated by their event labels  $\alpha$  and for each resultant state pair  $(q, \delta(q, \alpha))$ , a transition  $T_{q, \delta(q, \alpha)}$  is defined on  $G'_i$  such that  $I(T_{q, \delta(q, \alpha)}) = \{q\}$  and  $O(T_{q, \delta(q, \alpha)}) = \{\delta(q, \alpha)\}$ . Each such state machine Petri net  $G'_i$  thus models the flow of instances of  $G_i$  through its states.

##### (b) CSM Petri net synthesis:

To complete the plant synthesis, the thread subnets are composed to form a cascading state machine (CSM) Petri net representation by connecting the threads' Petri subnets via *spawning arcs* which model thread creation. Because the individual finite state automata do not have a built-in mechanism for representing the spawning of new threads, a simple mapping of events to thread creation is required. In the prototype provided, this is effected by a simple key/value mapping in a separate file called a *spawn map*. The keys are a subset of the events generable by the plant; the values are a subset of the start states of the individual thread automata—the restriction to start states is mandatory since instances of a given thread  $t$  logically always start in the same initial state  $q_{0,t}$ .

For each key/value mapping  $(\alpha, q_{0,t})$  in the spawn map, an output arc is added to each transition  $T_\alpha$  labeled with the event  $\alpha$  such that  $q_{0,t} \in O(T_\alpha)$ . The result is the CSM Petri net required, consisting of subnets corresponding to each underlying thread's finite state automaton connected by spawning arcs to model the creation of new thread instances—this Petri net is the plant  $G$  in Fig. 13. For instance, the CSM Petri net shown in Fig. 4 was created using the following spawn map:

$$\begin{aligned} (\text{CreateR}, q_{0, \text{Reader}}) &\equiv (\text{CreateR}, q_8) \\ (\text{CreateW}, q_{0, \text{Writer}}) &\equiv (\text{CreateR}, q_{12}) \\ (\text{CreateRW}, q_{0, \text{RW}}) &\equiv (\text{CreateR}, q_1). \end{aligned}$$

##### (c) Model insertion:

In the static case, a plant  $G$  and specification  $S$  are composed via the *supcon* operation to yield the controlled system, which can be used as a supervisor to control the plant. In the dynamic case, such a control code would be generated by parallel composition of the plant Petri net's reachability graph with that of each individual specification. It is the state space defined by this composition that the limited-lookahead algorithm traverses. Since it is not computed beforehand, the control code is not directly generated; instead, a system model  $M$  is defined simply to be the pair  $(G, \{S_i\})$ ;  $M$  is embedded in a *supervisor template* consisting of the supervisor firing mechanism and the lookahead algorithm (Fig. 12).

#### 5.5. Supervisor code translation

While at the interface to the controlled software the supervisor code is called in much the same way as in the static case, the dynamic supervisor differs drastically in implementation. The supervisor template implementation is pre-coded; translation consists of reading in the embedded model  $M$  and producing a representation of it in the target language. It is worth noting that  $M$  is “black-boxed” with respect to the supervisor; both the supervisor's firing mechanism and the lookahead algorithm act only on the interface provided by  $M$  for transitioning system state, and neither has any notion of its internal implementation.

#### 5.6. Supervisor call insertion

The final step is the same in the static and dynamic cases—the target code is re-parsed, and calls to the supervisor's firing mechanism (described below) are inserted at every code-point representing an event: both the controllable events associated with the instrumentation of step 1 (Section 5.1) and the uncontrollable events associated with control flow automatically generated by step 2 (Section 5.2). The supervisor code itself is linked (statically or dynamically, depending on the language) to the target code.

The next section outlines the action of the supervisor on the target code during execution.

## 6. Supervisor runtime action

At runtime, the sole interface of the target code with the supervisor is through the firing mechanism, typically implemented as a function/method call which takes as its sole argument the label of the prospective event to be fired and enacts control decisions based on the legality of the event. For clarity, we adopt the Java notation `Supervisor.fire(event)`

```

1  method supervisor.fire( $\alpha \in \Sigma$ )
2    supervisor.lock
3    // thread queues for and obtains supervisor lock
4    eventWasFired = false
5    while NOT eventWasFired
6       $q_{\text{current}} = \text{supervisor.currentNode}$ 
7      if  $M.\text{nextNode}(q_{\text{current}}, \alpha).\text{isBlocking}$ 
8         $\alpha.\text{wait}$ 
9      else
10       eventWasFired = true
11       supervisor.currentNode =  $M.\text{nextNode}(q_{\text{current}}, \alpha)$ 
12       lookahead( $q_{\text{current}}, N, \mathcal{L}(G), E$ )
13       for each  $\alpha \in \Sigma_c$  do
14          $\alpha.\text{notify}$ 
15     supervisor.unlock    // thread relinquishes supervisor lock

```

Fig. 15. Pseudocode for supervisor's fire method.

denoting the Supervisor object, its fire method and the argument *event*. Event labels are represented as simple tags in the String data type of the target language, where that exists; alternative data types can be used where appropriate. The event is the sole dynamic argument to the function, and the basis, in conjunction with the internal state of the model *M*, for the control decision; implied (unchanging) inputs are the plant and specification languages  $\mathcal{L}(G)$  and *E*, though the supervisor need have no direct access to their internals, and merely passes them to lookahead.

Fig. 15 provides pseudocode for the Supervisor's condition/lock mechanism by which threads are queued for event firing. The supervisor uses a single object lock with wait/notify conditions; one condition for each controllable event. Individual threads, upon reaching a specification-relevant code point, enact the Supervisor.fire(event) call, at which point an attempt is made to obtain the lock (line 2 of the code). If another thread is already engaged in a fire call, the current thread blocks.

Upon obtaining the lock, the thread consults the model (lines 5 and 6) to determine event legality (i.e., isBlocking). If the event requested is controllable and currently illegal, the thread is instructed to wait on the lock condition for that event (line 7). If legal, the event is fired, the next state is computed and the lookahead advanced; the thread then proceeds to send a notify signal to all the lock condition objects corresponding to controllable events, waking up any pending threads (lines 9 through 13) just before the thread releases the lock.

## 7. Implementation and initial validation

Initial implementations of the online-lookahead supervisor made use of a persistent tree, whose leaves were extended at each step. The general impression was that, online lookahead would be both time-intensive and time-critical, since control decisions would have to be made while minimizing the time during which threads would be bottlenecked while waiting for a decision. It quickly became apparent that the real limiting factor was space, not time. In the Readers/Writers example, explored to a lookahead depth of 10 controllable events, with a maximum of 10 readers and/or writers spawned by the code, the number of nodes in the lookahead at a given time varied enough to cause insufficient memory scenarios under the maximum RAM available in the environments at hand.

The solution was the elimination of persistent arcs in the lookahead tree for all but the top level of children—the level required for the control decision. Since node legality and blocking are accumulated from leaves to the first-level children, the remaining nodes would be released and garbage-collected on the backtracking phase of the recursion. The amount of RAM used went from over 3GB to roughly 81MB, making the exploration tenable in the space available. The time cost turned out to be manageable, with the only significant control decision delays resulting from those times in the execution window when many threads were engaging in critical sections simultaneously.

In general, the speed of lookahead is a critical factor, and one which showed itself to be a performance bottleneck for the Readers/Writers example once the space problem was solved. The space/time tradeoff was only partially investigated and warrants further research (see Section 9.1).

Preliminary testing was performed by modeling simple examples based on the Readers/Writers problem with template Java code. As the control-flow graph generation step was not fully implemented, care was taken to manually specify code and finite state automaton models which accurately reflected each other. The examples were sufficiently simple and transparent to bear manual scrutiny by the developer.

The examples simulated various concurrency scenarios such as mutual exclusion, precedence constraints and general condition-based waits. Constraint violation was simulated with Java assert statements which were only reachable if such violation occurred. The code examples were instrumented with random wait times between specification-critical code points to obtain broad coverage of different thread interleavings over multiple test iterations. All examples were written such that they would eventually terminate, with a bounded number of threads created.

The threads were designed to make constraint violation very likely in the absence of supervisory control. Indeed, when run with the supervisor deactivated, assertion exceptions were witnessed almost immediately in all executions. With the

supervisor activated and lookahead depths ranging up to  $N = 25$ , all executions ran to completion (the termination of all threads) without constraint violation. In order to check for violation, a separate `Assertion` object was implemented to track the parameters associated with the constraints. The `R` and `W` objects were instrumented with calls to the `Assertion` object to track the number of readers and writers in either a *pending* or *reading/writing* state (as per the description of mutex and priority constraints given in Section 2.2), and `assert` statements checking the `Assertion` attributes to raise exceptions in the same manner as the first example.

Again, when run with the supervisor deactivated, assertion exceptions invariably surfaced. With the supervisor activated, all executions ran to completion (the termination of all threads) without constraint violation. Lookahead length ranges up to  $N = 12$  were successful; beyond this, memory limitations caused the software to fail.

With a view to adding a further level of rigor to the testing process, a model-checking tool called Java PathFinder (JPF) [24] was brought to bear against the test examples. Java PathFinder consists of a specially-constructed host Java virtual machine (JVM) and a state-space search traversal mechanism which operates on target Java bytecode itself. The JPF JVM is capable of storing the relevant states in the execution path (trace) of the target code in a stack, and backtracking through states to explore alternative interleavings of code. The state space search can be exhaustive, depth-limited, and/or heuristically directed.

Exhaustive searches were attempted against the simple test examples described above. After a run of 44 days on a dual-core system with 4GB of RAM, a JPF test run of a simplified Reader/Writer ran out of memory; however, in the 44 days of state-space exploration, no deadlocks or constraint violations were found.

## 8. Related work

The literature on automatically generating parts of a software system from a formal specification is vast. Different versions of the problem have been considered in different research communities which has magnified differences in terminology and notation and complicated or prevented direct comparisons. Nonetheless, given the growing interest in using control systems theory to solve software engineering problems, a thorough discussion of how our work compares seems important.

We will use the following terminology: Let  $G$  be an open, reactive system, i.e., a collection of components that interact with each other and with some environment;  $G$  emits events, some controllable, some uncontrollable. Let  $E$  be a specification that restricts the occurrence of sequences of events. The controller synthesis problem is to automatically find a component  $S$  (called supervisor or controller), such that  $G||S$  satisfies  $E$ . Additionally,  $G||S$  may be expected to be deadlock free and  $S$  may have to be “optimal” or “minimally restrictive”. Note that,  $S$  may not exist as a separate component, because its control logic is directly integrated into  $G$  (e.g., in [25]); in these cases, the resulting system will also be referred to as  $G||S$ .

### 8.1. Approaches based on monitoring

In this class of approaches,  $S$  monitors the execution of  $G$  and reports violations of  $E$  or prevents them by only allowing a component in  $G$  to execute a controllable event if the occurrence does not cause  $E$  to be violated; otherwise, the component stays blocked. Enforced specifications are limited to safety properties. Work in this category ranges from, e.g., Campbell and Habermann’s Path Expressions [26,27] to “Monitoring Oriented Programming” by Meredith et al. [28]. A key characteristic of works in this class is that deadlock freedom is not guaranteed (in fact, by blocking a thread the controller  $S$  may introduce deadlock); moreover, no attempt at ensuring minimal restrictiveness is made. On the other hand, since no state space exploration takes place, dynamic concurrency can be supported.

### 8.2. Approaches based on static analysis

Similar to Andler [27], Deng et al. use counter variables to express specifications  $E$  and specify permissible concurrent behavior [25]; counter variables keep track of the number of processes inside certain user-specified code regions and global invariants enforce constraints on how processes can enter and exit these regions. Weakest pre-condition theory is used to compute appropriate guards for region entry and exit; these guards are inserted into  $G$  and block processes until they become true. As for monitoring-based approaches, dynamic concurrency is supported, but no guarantees for deadlock freedom or minimal restrictiveness are given. However, an integration of the approach with a software model checker is proposed to allow further validation of the code.

### 8.3. Approaches based on exploration of a finite state space

We distinguish between approaches that use DES techniques and those that do not. *Using DES techniques:* The work by Dragert et al. [1] has already been mentioned. Ramadge and Wonham’s classical, FSA-based DES control theory is used to generate a controller  $S$  for concurrent Java code  $G$  with a fixed number of threads that is known at compile time (static concurrency). User markup flags specification-relevant events in the code (all controllable) and an FSA-representation of  $G$  is automatically extracted from  $G$ ’s control flow graph (CFG). The generated supervisor information is used to create the supervisor  $S$ . Specifications are given as FSAs.

Work in the GADARA project uses Petri-net-based DES theory to generate controllers that guarantee deadlock freedom for concurrent code [11,10]. Similar to the work in this paper, the CFG of the code is translated into a Petri net. Then, a supervisory control method leveraging a structural analysis of the Petri net is applied. Using this method, deadlock freedom is characterized by the absence of empty siphons, i.e., a collection of places that, once empty, will remain empty forever. The controller prevents the siphons from becoming empty and is shown to be minimally restrictive. In contrast to [1] and this paper, only deadlock freedom (as opposed to more general properties) can be enforced. Although threads can be in different execution states, the number of threads in existence is determined prior to runtime and does not change at runtime.

In the following, approaches not explicitly using DES techniques are discussed, grouped by the formalism used for expressing the specification.

*Specification  $E$  as labeled transition system (LTS):* Tivoli and Inverardi cast the problem in a software architecture setting [29]. Their technique assumes that, the behavior of each component is specified as an LTS. Initially, these component specifications are composed using a Coordinator-Free Architecture (CFA); this system is then automatically transformed into a system with a Coordinator-Based Architecture (CBA) by synthesizing a coordinator that guarantees deadlock freedom and satisfaction of the coordination policies (also given as LTSS). Correctness (“system with CBA is indeed free of deadlock and policy violations”) and completeness (“all CFA executions free of deadlock and policy violations are possible in the CBA system”) are proven. The implementation uses the SYNTHESIS tool [30]; the synthesis algorithms used seem reminiscent of those used in DES, but a more detailed comparison is outside the scope of this paper.<sup>9</sup> Since all systems are assumed to be finite-state, dynamic concurrency does not seem to be supported.

Giannakopoulou et al. also assume that  $E$  and  $G$  are specified in terms of LTS (but represented as terms over the process algebra FSP) [31]. The Labeled Transition System Analyzer (LTSA) is used to perform a state space exploration and compute the “weakest environment”  $E$  that causes  $G$  to satisfy  $E$ . Since the state space is assumed to be finite, it appears that dynamic concurrency cannot be dealt with.

*Specification  $E$  as temporal logic:* A large body of work in formal specification and verification, program synthesis, realizability, and game theory uses some form of temporal logic to express system constraints. Most of this work is more theoretical in nature and papers present logics, algorithms or complexity results, rather than implementations. For instance, seminal work by Buechi and Landweber [32] showed that if  $G$  is a finite game and  $E$  is an  $\omega$ -regular language, then a finite winning strategy can always be constructed. Pnueli and Rosner later extend this work to LTL specifications [33], but also point out that the problem is 2EXPTIME-complete [34]. More recent work has identified an LTL subset which allows more efficient synthesis [35] and also produced tool implementations [36]. Note that, these papers are concerned with realizability and system synthesis (given a specification  $E$  over input and output sequences, find  $G$  such that  $G$  satisfies  $E$  for all input sequences) and thus address a slightly different problem. Jiang and Kumar study the control problem for CTL and CTL\* [37] and present a sound and complete controller synthesis algorithm for the control of CTL\* specifications. Complexity bounds for the synthesis and control of open environments with branching time temporal logic are given in [38]. Together with the work on module checking [39], this work is interested in analyses that check if the system  $G$  satisfies the specification in *all possible* environments. Both of these papers [38,39] thus address a slightly different problem from the one considered here which only asks for the existence of *at least one* environment (controller)  $S$  such that the specification is satisfied. In alternating-time temporal logic [40] properties such as “component  $A$  behaves correctly no matter how some other component  $B$  behaves” can be expressed and can also capture controller synthesis, realizability, and module checking.

Several approaches exist that solve the supervisor synthesis problem by means of existing model checkers. For instance, Gromyko et al. specify  $G$  in SMV and  $E$  in CTL and use NuSMV to obtain the controller. However, minimal restrictiveness is only guaranteed for a fragment of CTL. Similarly, the use of Spin for the generation of a controller for a concrete control application is described in [41]. On the theoretical side, Morgenstern and Schneider have shown how Ramadge and Wonham’s classical FSA-based DES control theory can be reduced to  $\mu$ -calculus model checking [42].

*Specification  $E$  as partial program:* In [43], an approach is presented in which developers can use a particular specification language to write partial, incomplete concurrent programs (called sketches) which express some partial insight into how a full implementation of a concurrent program should look; an automatic combinatorial synthesis procedure combined with a verification step that produces counter examples (such as model checking) then is used to iteratively complete the partial program such that given correctness criteria are satisfied. For instance, a sketch for the Dining Philosophers might leave the order in which forks are picked up and put down unspecified; the completion process would then find an order that results in a deadlock free program. The approach is proved correct, but no minimality assurance is given.

#### 8.4. Approaches based on exploration of a possibly infinite state space

We distinguish between approaches based on partial exploration and abstraction.

<sup>9</sup> In [29, p. 206], it is claimed that supervisor synthesis based on DES requires explicit specification of the deadlocking behaviors; this, however, is not the case.

**Table 1**

Comparison of most closely related work.

|                                                   | This paper                                           | [1]                                                               | [10]                                                                          | [25]                                                                        | [29]                                    | [52]                   | [50]                                                           |
|---------------------------------------------------|------------------------------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------------|------------------------|----------------------------------------------------------------|
| $G$ given as                                      | Code w/<br>markup                                    | Code w/<br>markup                                                 | Code                                                                          | Code w/<br>markup                                                           | LTS                                     | SMV code               | LTS                                                            |
| $E$ given as                                      | PN                                                   | FSA                                                               | n/a, only dead-<br>lock freedom                                               | In-equations<br>w/ counters                                                 | LTS                                     | CTL                    | Assertions                                                     |
| Technique to<br>compute $S$                       | Extract PN<br>from $G$ , use<br>limited<br>lookahead | Extract FSA<br>from $G$ , apply<br>supCon<br>operation<br>of [17] | Extract PN<br>from $G$ , $S$<br>prevents<br>siphons from<br>becoming<br>empty | Compute<br>weakest pre-<br>conditions to<br>guard region<br>entries & exits | Deadlock &<br>policy analysis<br>on LTS | CTL model<br>checking  | LTS explor. w/<br>atom. constr.<br>gen. & abstr.<br>refinement |
| Analysis<br>performed?                            | At run-time                                          | Before<br>run-time                                                | Before<br>run-time                                                            | Before<br>run-time                                                          | Before<br>run-time                      | Before<br>run-time     | Before run-<br>time                                            |
| Control<br>integrated<br>into $G$ or<br>separate? | Separate                                             | Separate                                                          | Separate                                                                      | Integrated                                                                  | Separate                                | Separate               | Integrated                                                     |
| $G  S$ deadlock<br>free?                          | Yes, in certain<br>cases                             | Yes                                                               | Yes                                                                           | No                                                                          | Yes                                     | No                     | ?                                                              |
| $S$ min. restric-<br>tive?                        | No                                                   | Yes                                                               | Yes                                                                           | No                                                                          | Yes                                     | Yes, for CTL<br>subset | Yes                                                            |
| Support for<br>dyn. conc.?                        | Yes                                                  | No                                                                | No                                                                            | Yes                                                                         | No                                      | No                     | No                                                             |

*Partial exploration:* The work presented in this paper falls into this category. Heymann and Lin [44] explored the concept of on-line control of discrete-event systems. In their approach, the supervisor's control decisions are computed only for the current stage of execution. In this manner, solutions may be nonoptimal but they are also more computationally efficient because the entire supervisor structure is not stored a priori. Chung et al. [4] were the first to formalize the DES control problem in the context of an online, partial exploration based on a limited lookahead policy. Different lookahead policies are compared with each other and with standard schemes based on the off-line generation and complete information. Grigorov and Rudie [20] apply this work to the problem of controlling dynamic DES, i.e., DES which can change arbitrarily and unpredictably at runtime. Lookahead policies are defined which include probabilities to capture the likelihood of a certain change and weight functions to be able to compare executions and obtain a notion of optimality. Both papers [4,20] provided the inspiration for using an online control scheme based on limited lookahead to control software with dynamic concurrency.

*Abstraction:* Most of the work found in this category uses abstraction to deal with time (e.g., [45–49]). An exception is recent work by Vechev et al. [50]. Here, the state space of programs over a simple, shared-variable concurrent language is searched for interleavings that lead to assertion violations. To improve scalability and to allow for programs with infinite state spaces to be handled, data abstraction is applied before the search. When an interleaving leading to an assertion violation is found, either the abstraction is automatically refined or suitable atomicity constraints are generated. If the analysis does not abort (in case assertion violations are unavoidable), does not exceed the available resources (due to size of program's state space), and does not run forever (the abstraction refinement process may not converge), the generated atomicity constraints are guaranteed to prevent assertion violations and be minimal (i.e., the removal of just one constraint would introduce violating interleavings). A key difference to our work is that the language considered in [50] is quite simple and does not allow dynamic concurrency or non-integer types. Also, specifications are expressed as assertions, but it is claimed that the approach could be extended to handle, e.g., temporal safety properties.

No approach was found that uses abstraction to deal with dynamic concurrency in the context of the supervisor synthesis problem.

### 8.5. Summary

Research on the automatic generation of code to control the execution of a system with respect to some specification has a long history with approaches building on concepts and techniques from a diverse set of domains including game theory, static analysis, state space exploration and control theory. Table 1 compares approaches which seem most related to ours. The approach presented in this paper is the only one that performs an online analysis and deals with dynamic concurrency, provides guarantees for deadlock freedom, and allows the use of general specifications. Apart from our own work in [20,151], where [51] is an early, preliminary articulation of the ideas underlying this paper that lacks a detailed description



of the lookahead algorithm and the supervisor generation, the work by the Gadara project [11,10], Deng et al. [25], Tivoli and Inverardi [29], and Vechev et al. [50] seems most closely related.

## 9. Conclusion

Discrete-event systems have already been shown to be an effective framework in which to autogenerate concurrency control code for high-level languages, given a set of assumptions including static foreknowledge of the processes (threads) to be controlled. The framework extension provided in this work accomplishes two goals:

- The accommodation of dynamic thread creation and termination, without such *a priori* knowledge as the numbers and lifetimes of individual threads;
- A solution to the general state-space tractability problem inherent to DES methods, in the form of online limited-lookahead search as a substitute to attempts at pre-calculation of impossibly large state spaces.

An important point to note is that, our approach is based on the control flow graph and is therefore, just like the work in [11,10], susceptible to being overly conservative due to infeasible paths in the control flow graph. In other words, if the control flow graph contains paths that can never get taken, then our approach may disable events unnecessarily, if one of these infeasible paths can lead to a specification violation.

To conclude, the framework extension retains the original guarantee of correctness in the absence of blocking wells as per Section 3.4, but does not guarantee maximum permissiveness. While the approach itself is fully automatic, it does assume that the user provides specifications of the desired code (plant) behavior and marks up the code with specification-relevant events. Given the complexity of concurrent programming, the effort required to create these inputs appears to have good potential to outweigh the costs.

A proof-of-concept implementation and its validation were described, and the limitations of the technique explored.

### 9.1. Future work

The technique given is envisioned as a framework upon which additional analysis can be built. In particular, much literature has been produced analyzing the structural properties of Petri nets which could be used to aid in the control decision process described herein (such as siphon analysis [11] or the theory of regions [53]). The application of such literature to this particular domain has also been left as future work.

While the basic validity of the technique was established, general performance evaluation was also left pending future work. Without such work, the general practicality of the technique is still an open question. In particular, the space/time tradeoff needs better measurement. Work by Winacott et al. [23], currently underway, makes use of the adjacency matrix of finite state automata and their eigenvalues to put estimates on the branching factor of a given lookahead tree, and may provide insight into the estimation of the required lookahead depth and by extension, the required system resources.

The issue of permissiveness is naturally related to the length of the lookahead: the shorter the lookahead, the more likely a harmless subtree (i.e., a subtree lacking in blocking traces) will be disabled due to lack of information on when a disabled event is re-enabled. A related issue is that of generability by the plant of a disabled event: a disabled event is entirely harmless if the event is not generable by the plant while it is disabled. Determining the generability of an event is non-trivial, but possible: the use of *siphon analysis* as in Wang and Lafortune [11] could be used to determine if a given disabled event is never generable by a plant in a given lookahead subtree. Siphon analysis was not implemented in our limited-lookahead prototype, and it is unclear if the added overhead would render the lookahead performance unacceptable. However, Wang and Lafortune can only determine if the event is forever nongenerable; not merely nongenerable for a specific window such as the duration of disablement.

As mentioned (Section 3.1), the only method by which the supervisor controls the plant is through the postponement of thread activity. Once a thread has adopted a particular control-flow path, the supervisor cannot change that path under the current paradigm. An avenue of research well worth investigation is the possibility of allowing the plant to react to control decisions to adopt alternative control-flow pathways. In a high-level language such as Java, this may take the form of a return value from the supervisor call; rather than a denied event transition resulting in simple blocking, a negative return value could indicate to the thread that the event is unavailable, allowing an alternative event to be attempted. This is equivalent to increasing the one-to-one mapping of available (generable) controllable events to threads. In addition, it may aid in the improvement of general concurrent performance, since threads unable to execute their initial choice of control flow can opt for an alternative processing pathway rather than simply blocking, decreasing the level of serialization due to concurrency control. We can think of such a control paradigm as *Supervisor-Aware Plant* (SAP) control. A supervisor which could handle both the current and SAP-driven event requests would be akin to other concurrency constructs, such as semaphores and locks, which can be accessed synchronously (blocking until available) or asynchronously (attempting to obtain without waiting). Indeed, a supervisor of this type might justifiably be seen as a generalization of more primitive concurrency locking mechanisms—one in which arbitrary lock retention rules are specifiable. Although fairly trivial to implement, the implications with respect to predictable plant behavior are unknown. Further research is required to characterize the effects of allowing awareness in the plant of supervisory decisions.



## References

- [1] C. Dragert, J. Dingel, K. Rudie, Generation of concurrency control code using discrete-event systems theory, in: Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE-16, 2008, pp. 146–157.
- [2] L. Holloway, B.H. Krogh, A. Giua, A survey of Petri net methods for controlled discrete event systems, *Journal of Discrete Event Dynamic Systems: Theory and Applications* 7 (2) (1997) 151–190.
- [3] Y. Wang, S. Lafortune, T. Kelly, M. Kudlur, S. Mahlke, The application of supervisory control to deadlock avoidance in concurrent software, in: Proceedings of the Workshop on Discrete Event Systems, Gothenburg, Sweden, 2008, pp. 287–292.
- [4] S.-L. Chung, S. Lafortune, F. Lin, Limited lookahead policies in supervisory control of discrete event systems, *IEEE Transactions on Automatic Control* 37 (12) (1992) 1921–1935.
- [5] J.L. Peterson, Petri nets, *ACM Computing Surveys* 9 (3) (1977) 223–252.
- [6] J. Dingel, K. Rudie, C. Dragert, Bridging the gap: Discrete-event systems for software engineering, in: Proceedings of International Conference on Computer Science and Software Engineering, C3S2E'09, ACM, 2009, pp. 67–71.
- [7] B. Cheng, R. Lemos, H. Giese, P. Inverardi, J. Magee, J. Andersson, B. Becker, N. Bencomo, Y. Brun, B. Cukic, G.M. Serugendo, S. Dustdar, A. Finkelstein, C. Gacek, K. Geihs, V. Grassi, G. Karsai, H. Kienle, J. Kramer, M. Litoiu, S. Malek, R. Mirandola, H. Müller, S. Park, M. Shaw, M. Tichy, M. Tivoli, D. Weyns, J. Whittle, Software engineering for self-adaptive systems, in: *Software Engineering for Self-Adaptive Systems: A Research Roadmap*, 2009, pp. 1–26. Ch.
- [8] W. Dahm, US Air Force Chief Scientist Report on Technology Horizons: A Vision for Air Force Science & Technology During 2010–2030, Tech. Rep. AF/ST-TR-10-01, US Air Force, 2010.
- [9] E. Denney, B. Fischer, Generating code review documentation for auto-generated mission-critical software, in: Third IEEE International Conference on Space Mission Challenges for Information Technology, SMC-IT, 2009, pp. 19–23.
- [10] Y. Wang, H. Cho, H. Liao, A. Nazeem, T. Kelly, S. Lafortune, S. Mahlke, S. Reveliotis, Supervisory control of software execution for failure avoidance: Experience from the gadara project, in: International Workshop on Discrete Event Systems, WODES'10, 2010.
- [11] Y. Wang, S. Lafortune, T. Kelly, M. Kudlur, S. Mahlke, The theory of deadlock avoidance via discrete control, in: Proceedings of the 36th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL'09, ACM, New York, NY, USA, 2009, pp. 252–263.
- [12] J.E. Hopcroft, R. Motwani, J.D. Ullman, *Introduction to Automata Theory, Languages and Computation*, second ed., Addison-Wesley, 2000.
- [13] C. Cassandras, S. Lafortune, *Introduction to Discrete Event Systems*, second ed., Springer, 2007.
- [14] W.M. Wonham, P.J. Ramadge, On the supremal controllable sublanguage of a given language, *SIAM Journal of Control and Optimization* 25 (3) (1987) 637–659.
- [15] L. Feng, W. Wonham, TCT: A computation tool for supervisory control synthesis, in: Proceedings of the 8th International Workshop on Discrete Event Systems, 2006, pp. 388–389.
- [16] K. Rudie, The integrated discrete-event systems tool, in: Proceedings of the 8th International Workshop on Discrete Event Systems, Ann Arbor, MI, 2006, pp. 394–395.
- [17] P.J. Ramadge, W.M. Wonham, Supervisory control of a class of discrete-event processes, *SIAM Journal of Control and Optimization* 25 (1) (1987) 206–230.
- [18] E.M. Clarke, O. Grumberg, D. Peled, *Model Checking*, MIT Press, 1999.
- [19] P.J. Courtois, F. Heymans, D.L. Parnas, Concurrent control with readers and “writers”, *Communications of the ACM* 14 (10) (1971) 667–668.
- [20] L.G. Grigorenko, K. Rudie, Near-optimal online control of dynamic discrete-event systems, *Journal of Discrete Event Dynamic Systems: Theory and Applications* (2006) 419–449.
- [21] M. Hack, Decidability questions for Petri nets, Ph.D. Thesis, Dept. of Electrical Engineering and Computer Science, M.I.T., 1976.
- [22] E.W. Mayr, An algorithm for the general Petri net reachability problem, in: Proceedings of the Thirteenth Annual ACM Symposium on Theory of Computing, STOC'81, ACM, 1981, pp. 238–246.
- [23] C. Winacott, B. Behinaein, K. Rudie, Methods for the estimation of the size of lookahead tree state-space, *Discrete Event Dynamic Systems* (2012) 1–21. <http://dx.doi.org/10.1007/s10626-012-0138-y>.
- [24] G. Lindstrom, P.C. Mehrlitz, W. Visser, Model checking real time Java using Java PathFinder, in: *Lecture Notes in Computer Science*, vol. 3707, 2005, pp. 444–457.
- [25] X. Deng, M. Dwyer, J. Hatcliff, M. Mizuno, Invariant-based specification, synthesis and verification of synchronization in concurrent programs, in: International Conference on Software Engineering, ICSE'02, 2002.
- [26] R.H. Campbell, A.N. Habermann, The specification of process synchronization by path expressions, in: *Symposium on Operating Systems*, 1974, pp. 89–102.
- [27] S. Andler, Predicate path expressions, in: Proceedings of the 6th ACM Symposium on Principles of Programming Languages, 1979, pp. 216–236.
- [28] P. Meredith, D. Jin, D. Griffith, F. Chen, G. Rosu, An overview of the MOP runtime verification framework, *International Journal on Software Techniques for Technology Transfer* (2011) 249–289. <http://dx.doi.org/10.1007/s10009-011-0198-6>.
- [29] M. Tivoli, P. Inverardi, Failure-free coordinators synthesis for component-based architectures, *Science of Computer Programming* 71 (3) (2008) 181–212.
- [30] M. Tivoli, M. Autili, Synthesis: a tool for synthesizing correct and protocol-enhanced adaptors, *Lobjet* 12 (1) (2006) 77–103 (special issue on Software Adaptation).
- [31] D. Giannakopoulou, C. Pasareanu, H. Barringer, Component verification with automatically generated assumptions, *Automated Software Engineering* 12 (2005) 297–320.
- [32] J. Buechi, L. Landweber, Solving sequential conditions by finite-state strategies, *Transactions of the American Mathematical Society* 138 (1969) 295–311.
- [33] A. Pnueli, Rosner, On the synthesis of a reactive module, in: *ACM Symposium on Principles of Programming Languages, POPL'89*, 1989.
- [34] R. Rosner, Modular synthesis of reactive systems, Ph.D. Thesis, Weizmann Institute of Science, 1992.
- [35] N. Piterman, A. Pnueli, Y. Saar, Synthesis of reactive(1) designs, in: *Proc. Verification, Model Checking, and Abstract Interpretation, VMCAI'06*, 2006, pp. 364–380.
- [36] B. Jobstmann, S. Galler, M. Weighhofer, R. Bloem, Anzu: a tool for property synthesis, in: *Computer Aided Verification, CAV'07*, 2007, pp. 258–262.
- [37] S. Jiang, R. Kumar, Supervisory control of discrete event systems with CTL\* temporal logic specifications, *SIAM Journal of Control and Optimization* 44 (6) (2010) 2076–2103.
- [38] O. Kupferman, P. Madhusudan, P. Thiagarajan, M. Vardi, Open systems in reactive environments: control and synthesis, in: 11th International Conference on Concurrency Theory, Concur'00, 2000.
- [39] O. Kupferman, M. Vardi, P. Wolper, Module checking, *Information and Computation* 164 (2) (2001) 322–344.
- [40] R. Alur, T. Henzinger, O. Kupferman, Alternating-time temporal logic, *Journal of the ACM* 49 (2002) 672–713.
- [41] M.M. Gallardo, P. Merino, L. Panizo, A. Linares, A practical use of model checking for synthesis: generating a dam controller for flood management, *Software: Practice and Experience* 41 (11) (2011) 1329–1347.
- [42] A. Morgenstern, K. Schneider, Using model checking to solve supervisor synthesis problems, in: *IEEE Conference on Decision and Control and European Control Conference, CDC-ECC'05*, 2005.
- [43] A. Solar-Lezama, C. Jones, R. Bodik, Sketching concurrent datastructures, in: *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI'08*, 2008.
- [44] M. Heymann, F. Lin, On-line control of partially observed discrete event systems, *Discrete Event Dynamic Systems* 4 (1994) 221–236.
- [45] J. Ostroff, Synthesis of controllers for real-time discrete event systems, in: *28th IEEE Conference on Decision and Control*, 1989.

- [46] H. Wong-Toi, G. Hoffman, The control of dense real-time discrete event systems, in: IEEE Conference on Decision and Control, 1991, pp. 1527–1528.
- [47] G. Behrmann, A. Cougnard, A. David, E. Fleury, K. Larsen, D. Lime, Uppaal-tiga: Time for playing games! (tool paper), in: International Conference on Computer-Aided Verification, CAV'07, 2007.
- [48] J. Jessen, J. Rasmussen, K. Larsen, A. David, Guided controller synthesis for climate controller using Uppaal Tiga, in: International Conference on Formal Modeling and Analysis of Timed Systems, FORMATS'07, 2007, pp. 227–240.
- [49] M. Mazo, A. Davitian, P. Tabuada, Pessoa: a tool for embedded controller synthesis, in: 22nd International Conference on Computer Aided Verification, CAV'10, 2010.
- [50] M. Vechev, E. Yahav, G. Yorsh, Abstraction-guided synthesis of synchronization, in: Proceedings of the 37th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL'10, ACM, 2010, pp. 327–338.
- [51] A. Auer, J. Dingel, K. Rudie, Concurrency control generation for dynamic threads using discrete-event systems, in: Proceedings of the 47th annual Allerton Conference on Communication, Control, and Computing, Allerton'09, IEEE, 2009, pp. 927–934.
- [52] A. Gromyko, M. Pistore, P. Traverso, A tool for supervisory control via symbolic model checking, in: 8th International Workshop on Discrete Event Systems, 2006.
- [53] M. Uzam, Synthesis of feedback control elements for discrete event systems using Petri net models and theory of regions, *International Journal of Advanced Manufacturing Technology* 24 (1) (2004) 48–69.